

Documentazione Progetto InfoLab

Giovanni Menicucci

June 2026

1 Introduzione

Questo documento tiene traccia dello sviluppo del progetto finale per il corso di Informatica con Laboratorio 2025/26, riguardante la soluzione dell'equazione del calore 2D.

2 Struttura del progetto

I task richiesti nel progetto vengono eseguiti con codici C++ (reperibili nella cartella "codice.cpp" o Python (nella cartella "codice_python"). Alcuni codici producono file di testo (nella cartella "dati"), che poi vengono letti dai codici successivi.

Le sezioni seguenti contengono i moduli principali che compongono il progetto.

3 Discretizzazione del dominio

Input: numero naturale N . Il dominio $[0, 1]^2$ è discretizzato con una griglia equispaziata di $(N + 2) \times (N + 2)$ punti.

Output: grafo di adiacenza, i cui nodi sono i punti interni alla griglia, con archi tra nodi primi vicini nella griglia.

File coinvolti: codice `discretizzazione.cpp` che genera un file `coords.txt` che rappresenta la griglia e un file `connectivity.txt` che rappresenta il grafo.

Strutture dati necessarie: `struct` per i nodi, per memorizzare indice progressivo n , indici della griglia i, j , coordinate spaziali x, y . `vector` per contenere la sequenza dei nodi interni, ordinati secondo l'indice n , per poi esportarli su `coords.txt`. `vector` di `vector` per la lista di adiacenza del grafo (lista e non matrice perché il grafo è sparso); e anche per costruire una matrice `griglia` che associ alle coordinate spaziali di ciascun nodo interno il relativo indice progressivo, e associ -1 ai nodi di bordo.

Funzioni:

- Generazione dei nodi con un doppio ciclo `for` che assegna indice progressivo (ordine lessicografico), coordinate nella griglia e spaziali. Questa funzione dovrebbe generare i singoli nodi, e riempire il vettore dei nodi interni e la matrice che associa alle coordinate l'indice progressivo. Complessità: $O(N^2)$, a causa del doppio ciclo `for`. All'interno di ogni iterazione si fanno solo operazioni elementari.

- Esportazione su file `coords.txt`. Questa funzione dovrebbe creare il file, scorrere sul vettore dei nodi interni e scriverli sul file, usando una riga per ciascun nodo, contenente n, i, j, x, y . Complessità: $O(N^2)$ (lunghezza del vettore su cui scorro).
- Generazione lista di adiacenza del grafo ed esportazione su file `connectivity.txt`. Dovrebbe generare un vector di N vector, ciascuno dei quali contiene i nodi collegati al nodo relativo; e creare il file txt. Uso un doppio ciclo for, sugli indici dei nodi interni. Uso la matrice `griglia` per verificare se i 4 vicini di un nodo sono di bordo o no, nel qual caso aggiungo alla lista di adiacenza e scrivo sul file l'indice progressivo dell'arco (cioè quanti archi ho creato fino a questo) e gli indici progressivi dei due nodi coinvolti.

4 Riordinamento dei nodi

Input: grafo definito sopra.

Output: nodi del grafo riordinati.

Implementazione: codice `riordinamento.cpp` che prende i file `coords.txt` e `connectivity.txt` generati nel modulo precedente, e genera un file `ordering.txt` che rappresenta la permutazione dei nodi.

5 Approssimazione dell'equazione differenziale con sistema lineare

Input: dominio dell'equazione differenziale discretizzato (ordinato nei due modi), coefficienti, funzione sorgente, condizione al bordo.

Output: matrice e termine noto del sistema lineare che rappresenta l'equazione.

Implementazione: codice `approx_lineare.cpp` che prende i file `coords.txt` (generato nel modulo "discretizzazione") e `ordering.txt` (generato nel modulo "riordinamento") e genera un file `A.txt` che rappresenta la matrice del sistema lineare, e un file `rhs.txt` che rappresenta il termine noto.

6 Fattorizzazione di Cholesky

Input: matrice A del sistema lineare.

Output: matrice L che fattorizza A . Immagini della struttura di A, L (entrate non nulle).

Implementazione: notebook `.ipynb` che legge `A.txt` generato nel modulo precedente ed esegue la fattorizzazione. Struttura della matrice (entrate non nulle).

7 Soluzione del sistema lineare

Input: fattorizzazione di A e termine noto del sistema lineare.

Output: soluzione del sistema lineare. Analisi del tempo necessario al variare di N e dell'ordinamento dei nodi. Plot della soluzione.

Implementazione: notebook `.ipynb` che legge la matrice L e `rhs.txt` e risolve il sistema lineare.